CS6886 Systems for Deep Learning — Assignment 5

Optimising Attention on a CPU

Saurabh Chatterjee (Roll No. NS26Z334)

November 6, 2026

Q1. My machine

I ran all experiments on a shared dual-socket server in the lab. Table 1 reproduces my `machine_spec.txt`.

CPU model	2 × Intel Xeon Gold 6338 @ 2.00 GHz (dual-socket server, 2 NUMA nodes)
Microarchitecture	Ice Lake-SP (Sunny Cove cores, 10 nm), family 6 model 106 stepping 6
Physical cores	64 (32 per socket × 2 sockets)
Logical cores (hardware threads)	128 (2-way Hyper-Threading)
Performance / efficiency cores	N/A (homogeneous: 64 identical cores, no P/E split)
Base clock (GHz)	2.0
Max single-core clock (GHz)	3.2 (Intel spec max turbo); measured 2.66 for scalar code and 2.38–2.40 for AVX-512 FMA code on one core (perf cycles / FMA microbenchmark)
All-core clock under load (GHz)	2.6 (Intel spec, non-AVX all-core turbo); measured 2.17 with AVX-512 FMA on all 64 cores
SIMD instruction set and width	AVX-512 (F/CD/BW/DQ/VL/IFMA/VBMI/VNNI), 512-bit = 16 FP32 lanes; also AVX2/FMA3 (256-bit) and SSE4.2
FMA units per core	2 × 512-bit FMA (ports 0+1 fused and port 5); verified: 64 FLOP/cycle measured
Vector registers per core	32 × 512-bit zmm (per hardware thread) + 8 opmask registers k0–k7
L1 data cache (per core)	48 KB, 12-way (L1i 32 KB)
L2 cache (size, shared by)	1.25 MB per core, shared by the 2 hardware threads of that core (80 MB total)
L3 cache (size, shared by)	48 MB per socket, shared by the 32 cores of a socket (96 MB total)
Cache line size (bytes)	64
RAM size (GB)	1024 GB (16 × 64 GB DIMMs; 1007 GiB visible to the OS)
RAM type, speed (MT/s), channels	DDR4-3200, 16 channels total (8 per socket: 4 IMCs × 2 channels, 1 DIMM per channel)
Operating system	CentOS Stream 8, Linux 4.18.0-348.2.1.el8_5.x86_64, glibc 2.28
Compiler and version	clang 22.1.8 (LLVM 22, from the ziglang 0.17.1 wheel, targeting glibc 2.28); OpenMP runtime: Intel libiomp5 2026.1.2 (libomp ABI); flags `-O3 -march=native`
Derived (calculations below)	
Peak FP32 GFLOP/s, one core	153.6 (2 FMA × 16 lanes × 2 FLOP × 2.40 GHz AVX-512 clock; measured 154.1)
Peak FP32 GFLOP/s, all cores	8888 (64 cores × 64 FLOP/cycle × 2.17 GHz; measured 8883). At the 2.0 GHz base clock: 8192
Memory bandwidth (GB/s)	409.6 theoretical (16 ch × 3200 MT/s × 8 B); measured STREAM triad 289.3 (all cores), 13.4 (one core)
Ridge point, one core (FLOP/B)	11.5 (153.6 / 13.4, the measured one-core bandwidth)
Ridge point, all cores (FLOP/B)	30.7 (8888 / 289.3, measured); 21.7 with the theoretical 409.6 GB/s

Table 1: Machine specification (contents of `machine_spec.txt`).

Measured clocks and bandwidth. AVX-512 code runs slower than the nominal turbo clock, so I measured the real numbers with two small microbenchmarks:

- `peak.c` runs 24 independent chains of `_mm512_fmadd_ps` on each thread. 24 chains are enough to hide the 4-cycle FMA latency on 2 ports. It reached 154.1 GFLOP/s on one core and 8883 GFLOP/s on 64 cores. `perf stat` separately reported 2.38 GHz for the one-core run. This result also shows that a core really has *two* 512-bit FMA units: with only one unit, the core would need to run at 154.1/32 = 4.8 GHz, which is impossible.

- `stream.c` is an OpenMP STREAM triad (`a[i] = b[i] + s*c[i]` on doubles). It uses arrays of 0.8–3.2 GB, first-touch initialisation and the best of 10 runs. It measured 13.4 GB/s on 1 core, 147.8 GB/s on one socket and 289.3 GB/s on all 64 cores (17.9, 197 and 386 GB/s if the write-allocate traffic is also counted).

Peak FP32 throughput. Each core can start 2 FMA instructions per cycle. Each FMA works on 16 floats and counts as 2 FLOPs (a multiply and an add), so one core does $2 \times 16 \times 2 = 64$ FLOP per cycle:

$$P_1 = 64 \ \tfrac{\text{FLOP}}{\text{cycle}} \times 2.40 \text{ GHz} = \mathbf{153.6\ GFLOP/s} \quad \text{(measured 154.1)}$$

$$P_{64} = 64 \text{ cores} \times 64 \ \tfrac{\text{FLOP}}{\text{cycle}} \times 2.17 \text{ GHz} = \mathbf{8888\ GFLOP/s} \quad \text{(measured 8883; at the 2.0 GHz base clock: 8192)}$$

In the rest of the report, "% of peak" always means a percentage of $P_1 = 153.6$ GFLOP/s.

Memory bandwidth. The theoretical bandwidth is 2 sockets × 8 channels × $3200 \cdot 10^6$ $\tfrac{\text{transfers}}{\text{s}}$ × 8 B = **409.6 GB/s** (204.8 GB/s per socket). With the STREAM triad I measured **289.3 GB/s** on all cores (71 % of the theoretical value) and **13.4 GB/s** on a single core. One core gets far less than the socket's bandwidth because it can only keep a limited number of cache misses in flight at once.

Ridge points. The ridge point is the peak compute divided by the bandwidth. I used the measured bandwidths:

$$I_1^* = \frac{153.6}{13.4} = \mathbf{11.5\ FLOP/B}, \qquad I_{64}^* = \frac{8888}{289.3} = \mathbf{30.7\ FLOP/B} \quad \text{(21.7 with the theoretical 409.6 GB/s).}$$

A kernel needs at least this many FLOPs per byte of DRAM traffic before it can be compute-bound.

Q2. Naive baseline

N	d	time (ms)	GFLOP/s	% of one-core peak
1024	64	164	1.64	1.07 %
2048	64	666	1.61	1.05 %
4096	64	2677	1.60	1.04 %
4096	128	9641	0.89	0.58 %

Table 2: Results of the naive implementation (`make naive`) on the four inputs. The percentage is relative to the one-core peak of 153.6 GFLOP/s.

phase	N=4096, d=64	N=4096, d=128
1. $S = QK^\top/\sqrt{d}$	947 ms (35 %)	2025 ms (24 %)
2. row-wise softmax	99 ms (4 %)	101 ms (1 %)
3. $O = SV$	1660 ms (61 %)	6494 ms (75 %)

Table 3: Breakdown of the naive run time into its three phases for the two $N = 4096$ inputs, as time per call and as a share of the total. I measured it by adding `clock_gettime` timers around the three loop nests in a copy of `attn_naive.c`.

How the time changes with N and d. When I double N, the time grows by a factor of 4.05 (1024→2048) and 4.02 (2048→4096). This matches the $O(N^2 d)$ work, and GFLOP/s stays almost constant at about 1.6. When I double d at $N = 4096$, the time should only double, but it grows by **3.6**×: throughput drops from 1.60 to 0.89 GFLOP/s. Table 3 shows where the extra time comes from. Phase 3 ($O = SV$) becomes 3.9× slower, while phase 1 ($QK^\top$) becomes only 2.1× slower, as expected.

Where the time goes. The naive code reaches only about 1 % of peak. I found four reasons:

1. **No vectorisation (up to 16× lost).** Both matrix products are written as dot products, `acc += a[k]*b[k]`. To vectorise a dot product, the compiler must change the order of the floating-point additions. Without `-ffast-math` it is not allowed to, so clang generates scalar `vfmadd231ss` instructions (I checked the assembly). Each one does 2 FLOPs, where a 512-bit FMA would do 32.

2. **A latency-bound dependency chain.** Every FMA in a dot product needs the result of the previous one, and an FMA takes 4 cycles. One dot product is therefore a serial chain of FMAs. The out-of-order core can overlap the end of one dot product with the start of the next, which is why phase 1 reaches about 2.3 cycles per FMA (0.88 ns) instead of 4. Even so, the two FMA ports could start 2 FMAs every cycle, so another factor of 4–5 is lost. `perf` reports only about 1 instruction per cycle.

3. **Walking down a column of V in phase 3.** The innermost loop runs over j and reads `V[j*d+k]`, which jumps $4d$ bytes (256 B or 512 B) at each step. Every load therefore touches a new cache line and uses only 4 of its 64 bytes. For $d = 64$, V is 1 MB and still fits in the 1.25 MB L2, so each FMA waits on an L2 access (1.55 ns per FMA, compared with 0.88 ns in phase 1). For $d = 128$, V is 2 MB, larger than L2, so the loads go to L3 (3.0 ns per FMA). This is why the time grows faster than linearly in d.

4. **The $N \times N$ matrix and the scalar `expf` are minor here.** At $N = 4096$, S is 64 MB, which is larger than the 48 MB L3, so it goes through DRAM: about 335 MB of traffic per call (Q5). Spread over 2.7 s, this is only 0.12 GB/s, far below the 13.4 GB/s a core can sustain. The N^2 scalar `expf` calls take 4 % of the time. Both become important once the matrix products are fast.

Q3. Optimisations (single thread)

Table 4 lists all variants and Figure 1 plots their times. `make check` reports PASS for every variant on every input; the largest relative error is 6.4×10^{-6}, well below the 10^{-4} limit. I also compared every variant against naive on awkward shapes ($N \in \{1, 33, 130, 500, 777, 1000\}$, $d \in \{16, 40, 48, 64, 80\}$) to test the padding and fallback paths.

variant	N	d	time (ms)	GFLOP/s	% of peak	speedup vs naive
`naive`	1024	64	164	1.6	1.1 %	1.0×
	2048	64	666	1.6	1.0 %	1.0×
	4096	64	2677	1.6	1.0 %	1.0×
	4096	128	9641	0.9	0.6 %	1.0×
`loops`	1024	64	22.2	12.1	7.9 %	7.4×
	2048	64	88.3	12.2	7.9 %	7.5×
	4096	64	381	11.3	7.3 %	7.0×
	4096	128	765	11.2	7.3 %	12.6×
`tiled`	1024	64	9.88	27.2	17.7 %	16.6×
	2048	64	39.6	27.1	17.7 %	16.8×
	4096	64	174	24.6	16.0 %	15.4×
	4096	128	236	36.3	23.7 %	40.8×
`fused`	1024	64	9.96	26.9	17.5 %	16.5×
	2048	64	39.2	27.4	17.8 %	17.0×
	4096	64	173	24.8	16.1 %	15.4×
	4096	128	280	30.7	20.0 %	34.5×
`best`	1024	64	2.80	95.7	62.3 %	58.5×
	2048	64	11.0	97.3	63.3 %	60.3×
	4096	64	44.0	97.6	63.5 %	60.8×
	4096	128	82.5	104.1	67.8 %	116.8×

Table 4: All single-threaded variants (fastest of 3 runs of the driver's median of 5, pinned to one core).

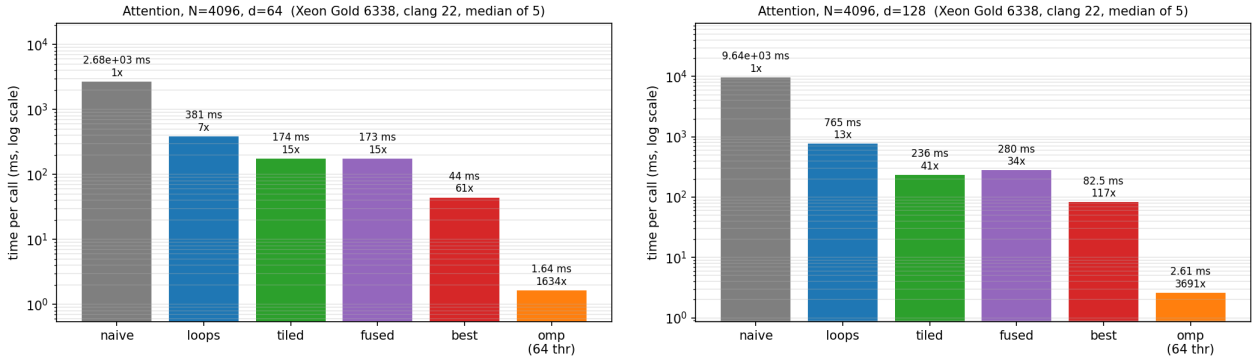


Figure 1: Time of every variant at $N = 4096$, on a log scale. The plots for all four inputs are in `plots/`.

(a) `attn_loops.c`: loop reordering — 7× faster

What I changed. First I transpose K once into $K^\top$ ($d \times N$). Phase 1 then runs in i-k-j order, `S[i][j] += (Q[i][k]*scale) * Kt[k][j]`, and phase 3 in i-j-k order, `O[i][k] += S[i][j] * V[j][k]`. In both cases the innermost loop now walks through two arrays with unit stride, and its iterations are independent of each other (an AXPY instead of a reduction). The compiler no longer needs to reorder any additions, so it vectorises both loops without `-ffast-math`. The `-Rpass=loop-vectorize` report confirms this ("vectorization width: 8, interleaved count: 4"). Phase 3 also no longer walks down a column of V. In the softmax I replaced the division by a multiplication with 1/sum. Finally, I added `restrict`, so that the compiler can drop most of its run-time aliasing checks.

Result. The variant reaches 11.2–12.2 GFLOP/s, which is 7.3–12.6× faster than naive. The speedup is largest at $d = 128$, because the slow strided access to V is gone. GFLOP/s no longer depends on d.

What limits it now (7–8 % of peak):

- *The AXPY loops are limited by loads and stores.* The accumulator `row[j]` lives in memory, so every vector FMA `row[j] += q*kt[j]` also needs 2 loads and 1 store. The core can sustain only about 1 such FMA per cycle. In `perf annotate`, the hottest instructions are the `vmovups` stores and the FMAs with memory operands.
- *The compiler uses 256-bit vectors.* When tuning for Ice Lake, clang prefers 256-bit vectors by default (its `prefer-vector-width` setting is 256), so the auto-vectorised code uses `ymm` registers. This alone limits the speed to half of the AVX-512 peak.
- *Data is reused only in L1/L2.* For every row i, the whole of $K^\top$ (1–2 MB) is streamed again from L2 or L3.
- *The softmax is still scalar.* `expf` takes 12 % of the time. The max and sum loops are reductions, so the compiler does not vectorise them either.

(b) `attn_tiled.c`: cache and register tiling — 15–41× faster

What I changed.

- *Register tiling.* I wrote two small kernels. The $QK^\top$ kernel computes a 6×32 block of S, and the PV kernel computes a 6×32 block of O. Each block is a plain-C array `float acc[6][32]` with loop bounds known at compile time. Clang unrolls these loops completely and keeps the block in 24 `ymm` registers for the whole reduction. The assembly confirms this: each kernel contains 24 register-only `vfmadd231ps` instructions. Every loaded K or V vector is now used by 6 FMAs, and every broadcast Q or P value by 4 vectors.
- *Packing.* K is copied into panels of 32 keys ($d \times 32$, contiguous in memory), and Q is multiplied by $1/\sqrt{d}$ in advance. Q, V and O are padded with zeros to whole multiples of the tile sizes, so the kernels never need special code for edges.
- *Cache tiling.* Q is processed in blocks of about 16 KB. A Q block and one 8 KB K panel then stay in L1 while the K panels stream past. For PV, V is processed in 32 KB blocks that stay in L1 while all row tiles use them.

Result. 24.7–36.3 GFLOP/s: about 2.2–3.2× faster than `loops` and up to 41× faster than naive.

What limits it now. The matrix products are no longer the problem. At $N = 4096$, $d = 64$, `perf` gives this split: `expf` 30 %, the scalar softmax loops 32 %, `qk_tile` 17 % and `pv_tile` 17 %. Each kernel therefore runs

at about 2.15 GFLOP/(0.17 × 174 ms) ≈ 73 GFLOP/s. That is 95 % of the 256-bit peak of 76.8 GFLOP/s, so the tiles themselves are close to optimal for the vector width that clang chose. The softmax costs $O(N^2)$ while the matrix products cost $O(N^2 d)$. This explains two observations. First, $d = 128$ reaches a higher GFLOP/s (36.3), because there the softmax is a smaller share of the work. Second, $N = 4096$ is slower than $N \leq 2048$: at $N = 4096$, S (64 MB) no longer fits in L3, so the softmax passes have to go to DRAM, while at $N \leq 2048$ ($S \leq 16$ MB) it stays in L3. Tiling improves data reuse inside the core, but it does not reduce DRAM traffic. S is still written, read, written again and read again.

(c) `attn_fused.c`: row-wise fusion — no faster than tiled

What I changed. The code takes 6 query rows at a time ($MR = 6$). For this strip it computes the scores against all N keys, applies the softmax, and multiplies by V, and only then moves on to the next strip. Only a $6 \times N$ strip of S exists at any moment (96 KB at $N = 4096$, which fits in L2); the full $N \times N$ matrix is never stored. I used a 6-row strip instead of a single row because 6 is the height of the register tile. The division by the softmax sum is now applied to the d outputs instead of the N probabilities. The kernels are the same as in (b), except that the PV tile now accumulates over all N keys in registers.

Result. 24.8–30.7 GFLOP/s. This is the same as `tiled` at $d = 64$, and slower at $d = 128$ (280 ms against 236 ms). Fusion did remove the DRAM traffic: `perf` counts 0.35 M L3 misses for `fused`, against 9.1 M for `tiled`, which is 26× fewer. The run time did not improve for two reasons:

- DRAM was never the bottleneck: `tiled` used only about 2 GB/s. The scalar softmax still dominates, with `expf` at 27 % and the max/sum loops at 29 % for $d = 64$.
- Fusing row by row removed the cache blocking of V. Each 6-row strip now streams the whole of V (1–2 MB), which does not fit in L1, and for $d = 128$ not even in L2. V is read 683 times per call. The share of `pv_tile` rises to 42 % at $d = 128$, which is why that case is slower than `tiled`.

One row at a time. I also tried literally processing one row at a time (`-DMR=1`). This gave only 9.2–15.5 GFLOP/s, 1.7–3× slower than the 6-row strip. With a single row, each loaded K or V vector is used by only one FMA, so the kernels are limited by loads. This experiment shaped the design of (d), which fuses over blocks of queries *and* blocks of keys.

(d) `attn_best.c`: FlashAttention-style AVX-512 kernel — 61–117× faster

My best version combines the following ideas.

1. **Blocking over queries and keys, with an online softmax (FlashAttention).** Queries are processed in blocks of $RB = 96$ rows and keys in blocks of BC rows. I set $BC = 8192/d$ (128 for $d = 64$, 64 for $d = 128$) so that one block of V is 32 KB and fits in L1. For each pair of blocks, the code computes $S = QK^\top$ ($96 \times BC$, kept in L1/L2), applies the softmax, and adds PV to O. Because the softmax is computed block by block, each row keeps a running maximum m and a running sum ℓ. When a new key block raises the maximum, the old O row and ℓ are rescaled by $2^{m_{\text{old}} - m_{\text{new}}}$. I fold this rescaling into the step where the PV kernel loads its accumulators, and I divide by ℓ only once, at the end. Compared with (c), each block of K and V is now reused by 96 query rows instead of 6, so K and V are read from L2/L3 only $N/96$ times.
2. **A vectorised exponential.** I multiply Q by $\log_2 e/\sqrt{d}$ in advance, so the softmax needs 2^x instead of e^x. I compute 2^x for 16 values at once. I split x into $x = n + f$ with $n = \text{round}(x)$ and $|f| \leq \frac{1}{2}$. Then 2^f comes from a degree-5 polynomial whose coefficients I fitted on Chebyshev nodes (maximum relative error 1.7×10^{-7}), and the `vscalefps` instruction multiplies the result by 2^n. This replaces glibc's scalar `expf`, which was the bottleneck in (b) and (c).
3. **AVX-512 register tiles written with intrinsics.** AVX-512 gives 32 `zmm` registers and twice the vector width of the auto-vectorised code. The $QK^\top$ tile covers 12 rows × 32 keys: 24 accumulators, and each step over k needs 2 loads of K and 12 broadcasts of Q for 24 FMAs. The PV tile covers 6 rows × 64 columns: 24 accumulators, and each key needs 4 loads of V and 6 broadcasts of P for 24 FMAs. With 24 independent accumulators, both FMA ports stay busy despite the 4-cycle latency.
4. **A softmax with a single pass over S.** The $QK^\top$ kernel computes the row maximum of each block while the scores are still in registers; masked `vmaxps` skips the padding keys beyond N. The running sum ℓ is kept as a vector of 16 partial sums. As a result, the softmax reads S only once per block and needs only one horizontal reduction per row.
5. **Packing.** K is packed into panels of 32 keys ($d \times 32$). Q is packed into panels of 12 rows, stored k-major ($d \times 12$), so the 12 values broadcast at one step over k sit in the same cache line. This also removes the per-row address arithmetic from the kernel. V is used in place, because its rows are already contiguous.

I also peeled off the first iteration over k (a multiply instead of an FMA into zero), after I noticed that clang was clearing the accumulator array through a `memset` on the stack.

How the result developed (GFLOP/s at $N = 4096$, for $d = 64$ / $d = 128$). The first version, with blocking, the vectorised exponential and intrinsics, used $BC = 256$ and $RB = 48$ and reached 84 / 89. Sweeping BC and RB raised this to about 95 / 100. The sweep showed that the block of V must fit in L1; $BC = 256$ was always the worst choice. The single-pass softmax, the degree-5 polynomial and the k-major packing of Q brought it to **97.6 / 104.1**. As a side experiment, I turned the softmax off (wrong output, for timing only); the kernels alone then reached 125 GFLOP/s, which showed that the softmax had cost about 20 % of the time before I optimised it.

Result. 95.7–104.1 GFLOP/s, i.e. **62–68 % of the single-core peak**. This is 61× ($d = 64$) and 117× ($d = 128$) faster than naive, and 4× faster than (b) and (c).

What limits it now. I profiled with the kernels forced out of line. At $d = 64$, $QK^\top$ takes 42 % of the time, PV 40 %, the softmax 11 % and everything else 5 %; at $d = 128$ the softmax takes 8 %. This means the kernels run at about 2.15 GFLOP/(0.42 × 44 ms) ≈ 115 GFLOP/s for $QK^\top$ and about 122 GFLOP/s for PV, i.e. 75–80 % of peak. The remaining gap has three causes:

- The kernels are short: d iterations for $QK^\top$ and BC for PV. The fixed cost of each call (24 stores and 48 masked maximum operations, or 24 accumulator loads, rescales and stores) is therefore noticeable.
- The kernels issue 10–14 loads for every 24 FMAs, plus address updates. These compete with the FMAs for the front end and the load ports.
- The softmax needs about 13 vector instructions for every 16 scores, while the matrix products need $2d/16 = 8$ FMAs per score. That is roughly 10 % extra work at $d = 64$ and 5 % at $d = 128$, which is why $d = 128$ runs faster.

DRAM bandwidth plays no role, since the inputs and the packed copies fit in L3 (see Q5). The fast path requires AVX-512F and a d that is a multiple of 16; otherwise the code falls back to a simple, portable scalar version.

Q4. OpenMP (`attn_omp.c`)

How I parallelised it. In the FlashAttention loop of `attn_best.c`, the query blocks are independent: each block owns its rows of O and its own softmax state. I distribute them over threads with `#pragma omp for schedule(dynamic,1)`. Each thread allocates its own S and O buffers inside the parallel region. Packing Q and K is parallelised as well. K, V and the packed arrays are shared but only read, so the code needs no locks and no reductions. On one thread the time is the same as `attn_best`, so the parallel version adds no overhead there. To give every thread enough work, I reduce the block height from 96 rows to $\lceil N/(4T) \rceil$, rounded up to a multiple of 12. For example, $N = 4096$ on $T = 64$ threads gives blocks of 24 rows. I compared this rule with several fixed block heights at 32 and 64 threads, and it was within measurement noise of the best one. All OpenMP runs use `OMP_PLACES=cores` and `OMP_PROC_BIND=close` (for 128 threads, `OMP_PLACES=threads`), and I again kept the best of 3 runs.

	N=1024, d=64			N=2048, d=64			N=4096, d=64			N=4096, d=128		
threads	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.
1	2.86	1.0	1.00	11.0	1.0	1.00	44.0	1.0	1.00	80.8	1.0	1.00
2	1.59	1.8	0.90	5.77	1.9	0.96	22.4	2.0	0.98	43.3	1.9	0.93
4	0.854	3.3	0.84	3.12	3.5	0.88	11.1	4.0	0.99	20.9	3.9	0.97
8	0.479	6.0	0.75	1.77	6.2	0.78	6.18	7.1	0.89	11.5	7.0	0.88
16	0.306	9.3	0.58	0.954	11.6	0.72	3.27	13.4	0.84	6.02	13.4	0.84
32	0.272	10.5	0.33	0.699	15.8	0.49	2.06	21.3	0.67	3.74	21.6	0.68
64	0.508	5.6	0.09	0.580	19.0	0.30	1.64	26.8	0.42	2.61	30.9	0.48
128	11.4	0.3	0.00	15.1	0.7	0.01	9.45	4.7	0.04	10.5	7.7	0.06
best	32 thr, 987 GF/s, 603×			64 thr, 1851 GF/s, 1147×			64 thr, 2622 GF/s, 1634×			64 thr, 3289 GF/s, 3691×		

Table 5: OpenMP scaling. Speedup is relative to 1 OpenMP thread, and parallel efficiency is speedup divided by the number of threads. The last row gives the best thread count for each input, the GFLOP/s it reaches, and its speedup over naive.

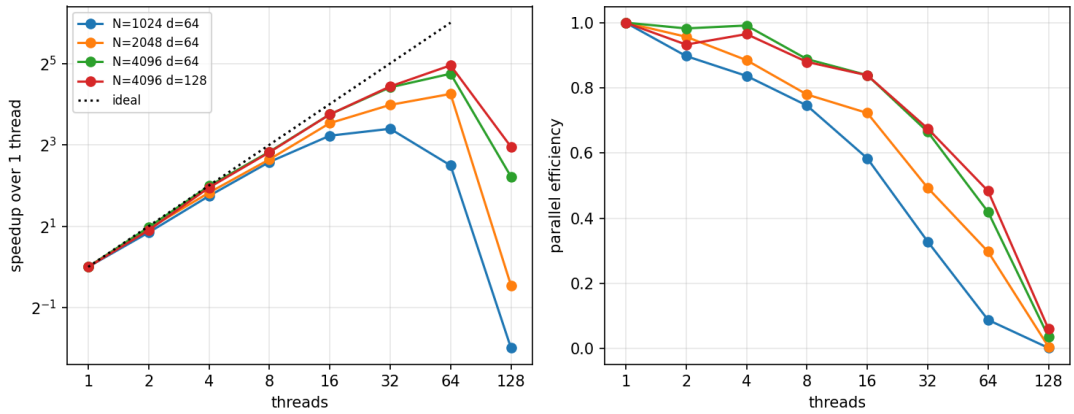


Figure 2: Speedup (left) and parallel efficiency (right) as the number of threads grows.

The scaling I observed. For the larger inputs, scaling is almost linear up to 4–8 threads (efficiency 0.88–0.99). Efficiency is 0.84 at 16 threads, 0.67 at 32 threads (one full socket) and 0.42–0.48 at 64 threads. The best results are **2622 GFLOP/s** at $N = 4096$, $d = 64$ (1.64 ms; 1634× naive; 29.5 % of the all-core peak) and **3289 GFLOP/s** at $d = 128$ (37 % of the all-core peak; 3691× naive). The smaller inputs stop improving earlier: $N = 1024$ is fastest at 32 threads, with 0.27 ms per call. Every thread count I tested passes `make check`. I see five reasons for the loss of efficiency:

1. **The clock drops when all cores are busy.** With AVX-512 running on every core, the clock falls from 2.40 to 2.17 GHz (Q1). This alone limits efficiency to about 0.90 at 64 threads.
2. **There are too few blocks to share out evenly.** At $N = 4096$ with 64 threads there are 171 blocks of 24 rows, about 2.7 per thread. The slowest threads must do 3 blocks, so efficiency cannot exceed 0.89. At $N = 1024$ there are only 86 blocks of 12 rows for 64 threads (efficiency at most 0.67), and each block lasts only a few microseconds. Smaller blocks also reduce the reuse of K and V. Every block reads the whole of K and V from L3, so the L3 traffic grows as the blocks shrink. At 64 threads it is 342 MB per 1.6 ms call, about 210 GB/s across the two L3 caches.
3. **Each call has fixed costs.** Every call starts and joins the thread team, passes two implicit barriers (after packing and at the end), and allocates buffers in every thread. These costs do not shrink as threads are added, while the whole call lasts only 0.3–2 ms. `perf` shows that the total number of instructions grows from 1.71 G on 1 thread to 2.49 G on 64 threads for the same work. The extra 46 % is threads spinning at OpenMP barriers while they wait for the slowest thread.
4. **The machine has two sockets.** With `close` binding, threads 1–32 run on socket 0 and threads 33–64 on socket 1. The threads on socket 1 read the shared inputs and the packed K from the other socket's memory and L3, over the UPI link. Efficiency drops from 0.67 to 0.42–0.48 exactly at this boundary.
5. **The machine is shared, and hyper-threading does not help.** Other users' jobs kept about 8–10 logical CPUs busy. With 128 threads, a pinned thread that shares its CPU with another process is time-sliced in millisecond steps, and all other threads wait for it at the barrier. The result is 9–15 ms, which is slower than one thread. I checked this explanation directly: 128 pinned threads took 72 ms, 128 unpinned threads 6.9 ms, 120 unpinned threads 3.1 ms and 96 threads with `spread` binding 1.84 ms. Even on an idle machine, hyper-threading would not help this kernel: the two threads of a core share its 2 FMA units, and they share the L1 and L2 that the block sizes were chosen for.

Q5. Roofline

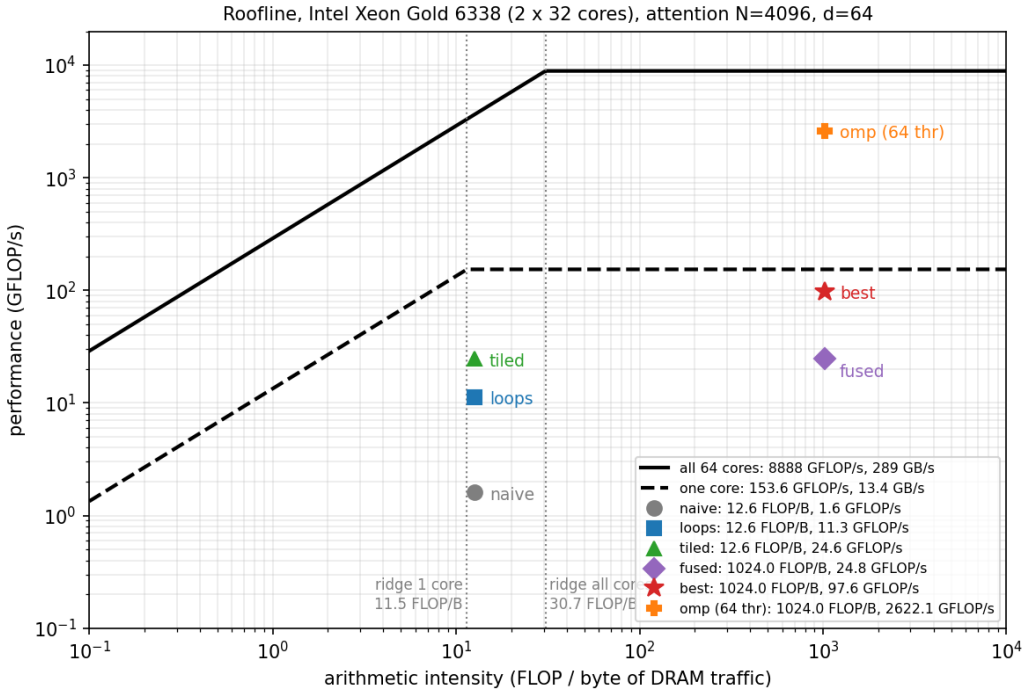


Figure 3: Roofline of the Xeon Gold 6338, built from the Q1 numbers (one core: 153.6 GFLOP/s and 13.4 GB/s; all cores: 8888 GFLOP/s and 289.3 GB/s), with every variant placed at $N = 4096$, $d = 64$. The omp point uses its best thread count (64).

How I computed the arithmetic intensity. For the FLOPs I used the driver's count, $4N^2d = 4.295$ GFLOP, which ignores the exponentials. For the bytes I counted DRAM traffic per call with a simple model based on the loop structure of each variant. I then checked the model against the L3-miss counter of `perf`, which is called `longest_lat_cache.miss`.

- **Traffic every variant must have:** reading Q, K, V and writing O costs $16Nd = 4.19$ MB.
- **naive, loops and tiled** also send S through DRAM, because at $N = 4096$ it is 64 MB, larger than the 48 MB L3. Phase 1 writes S ($4N^2$ bytes, plus another $4N^2$ for the write-allocate reads). The softmax reads and writes every row once ($8N^2$); a row is only 16 KB, so it stays in L1 across the three softmax passes. Phase 3 reads S once more ($4N^2$). The total is $20N^2 = 335.5$ MB, so

$$I = \frac{4N^2d}{16Nd + 20N^2} = \frac{4.295 \cdot 10^9}{339.7 \cdot 10^6} = \mathbf{12.6\ FLOP/B}.$$

  The transposed or packed copies of K in loops and tiled add less than 1 %. All three variants therefore have the same intensity: reordering and tiling change how data is reused on the chip, not how much reaches DRAM. For $N \leq 2048$, S fits in L3, so there these variants actually move less data than the model says.

- **fused, best and omp** never write S to memory; a strip or block of it stays in L1/L2. The packed copies of Q and K (and of V and O in fused) are 1 MB each and stay in L3, together with the 3 MB of inputs. Only the unavoidable traffic is left:

$$I = \frac{4N^2d}{16Nd} = \frac{N}{4} = \mathbf{1024\ FLOP/B}.$$

- **Check with perf** (L3 misses × 64 B, per call): loops 101 MB, tiled 97 MB, fused 3.7 MB, best 2.6 MB. As the model predicts, the fused variants move 26–39× less data. The measured values for loops and tiled are lower than the model, because write-backs are not counted and part of S still hits in L3.

What limits each variant.

- **naive** (12.6 FLOP/B, 1.6 GFLOP/s, 1 % of the roof). It sits just to the right of the one-core ridge (11.5), so its roof is the compute peak. It is 100× below that roof, so it is limited by the core and not by memory: scalar code, serial FMA chains, and L2/L3 latency from the strided walk over V (Q2).
- **loops** (12.6, 11.3 GFLOP/s) and **tiled** (12.6, 24.6 GFLOP/s) have the same intensity and are also far below the roof, so they too are limited by the core. In loops the bottleneck is the load/store-bound 8-wide AXPYs; in tiled it is the scalar `expf` and softmax loops. Their intensity would still matter in a parallel version. 12.6 FLOP/B is to the left of the all-core ridge (30.7), so any multi-threaded version that stores S could never exceed $289.3 \times 12.6 \approx 3.6$ TFLOP/s (41 % of peak): storing S would make it memory-bound.
- **fused** (1024, 24.8 GFLOP/s). Fusion moves the variant 80× to the right, deep into the compute-bound region, but its speed does not change. This shows that memory traffic was never the limit on one core. The limits are inside the core: the scalar exponential, and V (and K) being streamed again for every 6-row strip. Measured against L3 traffic, fused does only about 4.3 GFLOP/1.4 GB ≈ 3 FLOP/B. best does about 50 FLOP/B (86 MB of L3 traffic per call), because it reuses each block of K and V for 96 rows.
- **best** (1024, 97.6 GFLOP/s, 64 % of the one-core roof). It is compute-bound. The remaining gap comes from the in-core overheads described in Q3(d): the softmax instructions, the fixed cost of each kernel call, and pressure on the load ports and address arithmetic.
- **omp** (1024, 2622 GFLOP/s, 29.5 % of the all-core roof). It is still in the compute-bound region: it moves about 4 MB per 1.64 ms call, roughly 2.6 GB/s, far below 289 GB/s. The gap to the roof is not caused by memory. It comes from the parallel effects described in Q4: the lower all-core clock, uneven work per thread, barriers and thread start-up, traffic between the sockets, and other users' jobs on the shared machine.

Summary. At $N = 4096$, $d = 64$: naive 2677 ms → loops 381 ms → tiled 174 ms → fused 173 ms → best 44.0 ms (61× faster, 64 % of peak, single thread) → omp 1.64 ms on 64 cores (1634× faster than naive).

Appendix: reproducing the results

```
export PATH=$HOME/tools/bin:$PATH # clang 22.1.8 wrapper (zig cc) + libiomp5 for -fopenmp
for v in naive loops tiled fused best; do taskset -c 10 make $v; done
for t in 1 2 4 8 16 32 64; do OMP_NUM_THREADS=$t OMP_PLACES=cores OMP_PROC_BIND=close make omp; done
make check
```

I kept the Q1 microbenchmarks (`peak.c`, `stream.c`), the sweep scripts and the plotting script outside the submission folder, so that the repository has exactly the structure the handout asks for. The core of `peak.c` is:

```
#pragma omp parallel
{ __m512 a[24], b = _mm512_set1_ps(0.999999f), c = _mm512_set1_ps(1e-7f); /* 24 independent chains */
    for (long it = 0; it < iters; it++)
        for (int i = 0; i < 24; i++) a[i] = _mm512_fmadd_ps(a[i], b, c); /* fully unrolled */
} /* FLOP = 2 * 16 * 24 * iters * threads */
```